



COVENANT UNIVERSITY
CANAANLAND, KM 10, IDIROKO ROAD
P.M.B 1023, OTA, OGUN STATE, NIGERIA.
B.SC DEGREE EXAMINATION

COLLEGE: Science & Technology SCHOOL: Natural & Applied Sciences
DEPARTMENT: Computer & Information Sciences SESSION: 2014/2015
SEMESTER: Alpha COURSE CODE: CSC213
CREDIT UNIT: 3 COURSE TITLE: Structured Programming
INSTRUCTION: Instruction(s): Answer ANY FOUR Questions
TIME: 3 HOURS

1. a) Briefly explain the main problem of computer programming addressed by Structured Programming

3½ Marks

- b) State the three main features of structured programming. Describe the fundamental ideas in each of them.

9 Marks

c)

Compare and contrast between the Structured Programming and Object Oriented Programming.

5Marks

2. a. Explain in details the concept of Modularity in structured programming.

3½ Marks

- b. Give a detailed overview of python programming State four application domains with examples of Python programming language.

8 Marks.

- c. Briefly state with appropriate examples, the types of Control Structures in Python Programming.

6 Marks

3. a) Write a python program to calculate the factorial of a positive integer quantity. The program should call another function called factorial.

7½ Marks

- b) What does the following function do?

```
def find (strng, ch):
```

```
    index = 0
```

```
    while index < len (strng):
```

```
        if strng [index] == ch:
```

```
            return index
```

```
        index += 1
```

```
    return -1
```

3Marks

- c) Define a string S of four characters: S = "spam". Write an assignment that changes the string to "slam", using only slicing and concatenation.

4Marks

d) Fill in the blank spaces;

```
>>> s = "Peter, Paul, and Mary"
>>> print s [0:5]
.....
>>> print s [7:11]
.....
>>> print s [17:21]
.....
```

3Marks

4. a) Write a python program that adds two values gotten from two different functions.
- The first function should calculate the sum of even numbers between 1 and 100.
 - The second function should calculate the sum of odd numbers between 1 and 100
 - In the main function, calculate and print the sum of the results from the first two functions.

7½Marks

b) Find the error in the following code segment and provide the correct code segment:

```
def username_pin()
    database = [
        ['adesuwa', '1234'],
        ['innocent', '4242'],
        ['smith', '7524'],
        username = rawinput('User name: ')
        pin = rawinput('PIN code: ')
        if [usernme, pin] in database
            Print 'Access granted'
        else:
            print 'Access denied')
```

6Marks

c) Explain and give examples of the following;

- i. Lists
- ii. Strings

4Marks

5. a) Write a function named `is_divisible_by_3` that takes a single integer as an argument and prints “This number is divisible by three.” if the argument is evenly divisible by 3 and “This number is not divisible by three.” otherwise.

Now write a similar function named `is_divisible_by_5`.

7½ Marks

b) Give examples of the following statements in python;

- i. If statement
- ii. The if statement with alternative execution
- iii. Chained conditional statements

iv. While statements

6Marks

c) What will be the output of the following programs?

i. `>>> a_list = ['a', 'b', 'c', 'd', 'e', 'f']
>>> a_list[1:3] = []
>>> print a_list`

ii. `>>> fruit = ["banana", "apple", "quince"]
>>> fruit[0] = "pear"
>>> fruit[-1] = "orange"
>>> print fruit`

iii. `>>> range(10)`

iv. `>>> range(1, 10, 2)`

4Marks

6. a) Write a python program that will accept the students score in a course and print the letter grade as follows:

- 70 and above: A
- 60-69 B
- 50-59 C
- 45-49 D
- Below 45 F

Note: make sure you take care of scores less than 0 and greater than 100

5Marks

b) i. What two common syntaxes are required in a C-like language, but omitted in Python?

2Marks

ii. How are the statements in a nested block of code normally associated in Python?

2Marks

c) What does the following program do.

```
infile = open('test2.txt', 'r')  
outfile = open('test.txt', 'w')  
while True:  
    text = infile.read()  
    if text == "":  
        break  
    outfile.write(text)  
infile.close()  
outfile.close()
```

3½Marks

d) Write a python program to calculate the sum of the first 10 terms of the series.

$1/1 + 1/4 + 1/9 + 1/16 + \dots$

5 Marks

MARKING SCHEME

1. (a) State and explain the main problem of computer programming addressed by structured programming?

- ▶ To minimize Program complexity. **(1.5Marks)**
- ▶ Most programs that do anything significant in the real world are rather long .i.e. word processor, OS, Compiler, Enterprise software- banking software. They have complicated interrelated parts and contain several thousand or even millions lines of codes which makes them very difficult to understand, maintain and reuse. **(2Marks)**

1. (b) State the three main features of structured programming?

Describe the fundamental ideas in each of them.

Three main aspects:

- ▶ Top-down analysis for problem solving
- ▶ Modularization for program structure and organization,
- ▶ Structured code for the individual modules. **(3Marks)**

Description:

- ▶ Top down analysis is a method of problem solving. The essential idea is to subdivide a large problem into several tasks or parts. Top down analysis, therefore, simplifies or reduces the complexity of the process of problem solving.
- ▶ Modular programming is a method of organizing these instructions. Large programs are broken down into separate, smaller sections called modules, subroutines, or subprograms. Each module has a specific job to do and is relatively easy to write.
- ▶ Structured codes: In SP, are organized within various control structures. A control structure represents a unique pattern of execution for a specific set of instructions. It determines the precise order in which that set of instructions is executed. **(6Marks)**

- ▶ **1.(c) Compare and contrast between the Structured Programming and Object Oriented Programming** **(5Marks)**

Structured Programming	Object Oriented Programming
Structured or procedural programming is a programming paradigm that attempts to divide the problem into smaller blocks or	OOP is a programming paradigm that program is written as a collection of interacting objects.

procedures which interact with other.	
In structured programs the blocks are pieces of code which are executed as the program is run	In object-oriented programs objects have lives of their own – they can be created, copied, destroyed or even lost!
Structured programming is a programming technique in which the programmer divides the source code into logical chunks of code. Using structured programming techniques, programs are typically organized around code. This approach can be thought of as “code acting on data.”	Object-oriented programming is a programming technique in which programs work the other way around. They are organized around data, with the key principle being “data controlling access to code.” In an object-oriented language, you define the data and the routines that are permitted to act on that data. Thus, a data type defines precisely what sort of operations can be applied to that data.
In Structured programming, the program is built around functions and /or modules which are nothing but reusable pieces of programs. Typically it follows a top-down design approach with block-oriented structures.	The object-oriented programming paradigm is built on the foundation laid by the structured programming concept. Object-oriented programming took the best ideas of structured programming and combined them with several new concepts. In object-oriented programming, the program is built around objects which combine data and functionality.

In Structured programming, the program is built around functions and /or modules which are nothing but reusable	The object-oriented programming paradigm is built on the foundation laid by the structured programming concept. Object-oriented programming took the best ideas of structured
---	---

pieces of programs. Typically it follows a top-down design approach with block-oriented structures.	programming and combined them with several new concepts. In object-oriented programming, the program is built around objects which combine data and functionality.
In structured programming large programs are broken into smaller pieces called subprograms. Each subprogram can have its own data and logic. The basic composition of any type of structured programming includes three fundamental elements: sequencing, selection and repetition.	OOP provides a new way of organizing data and code that promises increased control over the complexity of the software development process. The major difference is that OOP treats data as critical elements (as it should be) in the program development and does not allow it to flow freely around the system. In OOP the focus is not on the code but on what you want the code to do. Basic concepts of OOP: object, class, inheritance, methods, Encapsulation and polymorphism.

2. (a) Explain in details the concept of Modularity.

(3.5Marks)

- ▶ It is a method of organizing computer instructions. Large programs are broken down into separate, smaller sections called “*Modules, Subroutines, or Subprograms*”. *Each module has a specific task/operation to execute which is relatively less complexity to achieve.*

2. (b) Give a detailed overview of Python Programming. State four application use cases of python, stating two (2) examples in each case.

(8Marks)

- ▶ A versatile, interpreted, high-level, & dynamic programming language.
- ▶ Multi-paradigm: Simple procedural programming, OOP & Functional programming.
- ▶ Easy to Learn: Automatic memory mgt, high-level built-in data structures & libraries.
- ▶ Highly Readable: Intuitive syntax, Interpreted language, dynamic typing, etc.

- ▶ Portable Language: Many Versions-CPython, Jython, IronPython, PyPy
- ▶ Extensibility: Reusable Codes-Module & Packages, Open Source, Modularity.

Application Areas:

- ▶ Application development, e.g 3D games, GUI-based applications, etc
- ▶ Web development, e.g Youtube, Reddit, Yelp, etc
- ▶ Scientific computing, e.g Bioinformatics, NLP, machine learning, etc.
- ▶ Scripting, e.g OpenOffice, Games, Emacs, Blender, etc.

2. (c) Briefly state with appropriate examples the types of Control

Structures in Python Programming. (6Marks)

- ▶ Conditionals: if Stmts, elif/else Stmts, *No Switch Stmt in Python.
- ▶ Loop: while Stmts.
- ▶ Continue and Break Statements.

Examples:

if & elif Stmts.

```
if conditionExpr1: statement1
elif conditionExpr2: statement2
elif conditionExpr2: statement2
... ..
else: statement4
```

Examples:

while Stmt.

```
while conditionExpr: statement
```

Examples:

continue & break Stmts.

```
Expr while conditionExpr: statement
    if conditionExpr1: continue statement1
    if conditionExpr2: break
```

**3. .a) Python program to display the Fibonacci
sequence up to n-th term using**

```

# recursive functions

def recur_fibo(n):
    """Recursive function to
    print Fibonacci sequence"""
    if n <= 1:
        return n
    else:
        return(recur_fibo(n-1) + recur_fibo(n-2))

# take input from the user
nterms = int(input("How many terms? "))

# check if the number of terms is valid
if nterms <= 0:
    print("Plese enter a positive integer")
else:
    print("Fibonacci sequence:")
    for i in range(nterms):
        print(recur_fibo(i))

```

b) find is the opposite of the [] operator. Instead of taking an index and extracting the corresponding character, it takes a character and finds the index where that character appears. If the character is not found, the function returns -1.

c)

- >>> S = "spam"
>>> S = S[0] + 't' + S[2:]
>>> S
'slam'
- >>> S = S[0] + 't' + S[2] + S[3]
>>> S
'slam'

d)

```

>>> s = "Peter, Paul, and Mary"
>>> print s [0:5]
Peter
>>> print s [7:11]
Paul
>>> print s [17:21]
Mary

```

4. a)

```

def sumEven( ):
    sum_Even=0
    i=0;
    while i<100:
        sum_Even=sum_Even +i
        i=i+2
    return sum_Even

def sumOdd( ):
    sum_Odd=0
    i=1;
    while i<100:
        sum_Odd=sum_Odd +i
        i=i+2
    return sum_Odd

totalSum = sumOdd() + sumEven();
print (totalSum)

```

b)

```

def username_pin():
    database = [
        ['adesuwa', '1234'],
        ['innocent', '4242'],
        ['smith', '7524'],
    ]
    username = raw_input('User name: ')
    pin = raw_input('PIN code: ')
    if [username, pin] in database:
        print 'Access granted'
    else:
        print 'Access denied'

```

c) Lists: The Python list object is the most general sequence provided by the language. Lists are positionally ordered collections of arbitrarily typed objects, and they have no fixed size. They are also mutable—unlike strings, lists can be modified in-place by assignment to offsets as well as a variety of list method calls.

Strings: strings are *immutable* in Python—they cannot be changed in-place after they are created. For example, you can't change a string by assigning to one of its positions, but you can always build a new one and assign it to the same name. Strings are qualitatively different from the other four because they are made up of smaller pieces — **characters**.

5. a)

```
def is_divisible_by_3(n):
    if (n%3==0):
        print("This number is not divisible by 3")
    else:
        print("This number is not divisible by 3")
```

```
def is_divisible_by_5(m):
    if (m%5==0):
        print("This number is divisible by 5")
    else:
        print("This number is not divisible by 5")
```

b)

i.

if BOOLEAN EXPRESSION:

STATEMENTS

e.g. if $x > 0$:

```
print ("x is positive")
```

ii.

A second form of the if statement is alternative execution, in which there are two possibilities and the condition determines which one gets executed. The syntax looks like this:

```
if x % 2 == 0:
```

```
print ("x is even")
```

```
else:
```

```
print ("x is odd")
```

iii

Sometimes there are more than two possibilities and we need more than two branches. One way to express a computation like that is a **chained conditional**:

```
if x < y:
```

```
print x, "is less than", y
```

```
elif x > y:
```

```
print x, "is greater than", y
```

```
else:
```

```
print x, "and", y, "are equal"
```

iv.

While Statement

def sequence(n):

while n != 1:

print n,

if n % 2 == 0: # n is even

n = n / 2

else: # n is odd

n = n * 3 + 1

c)

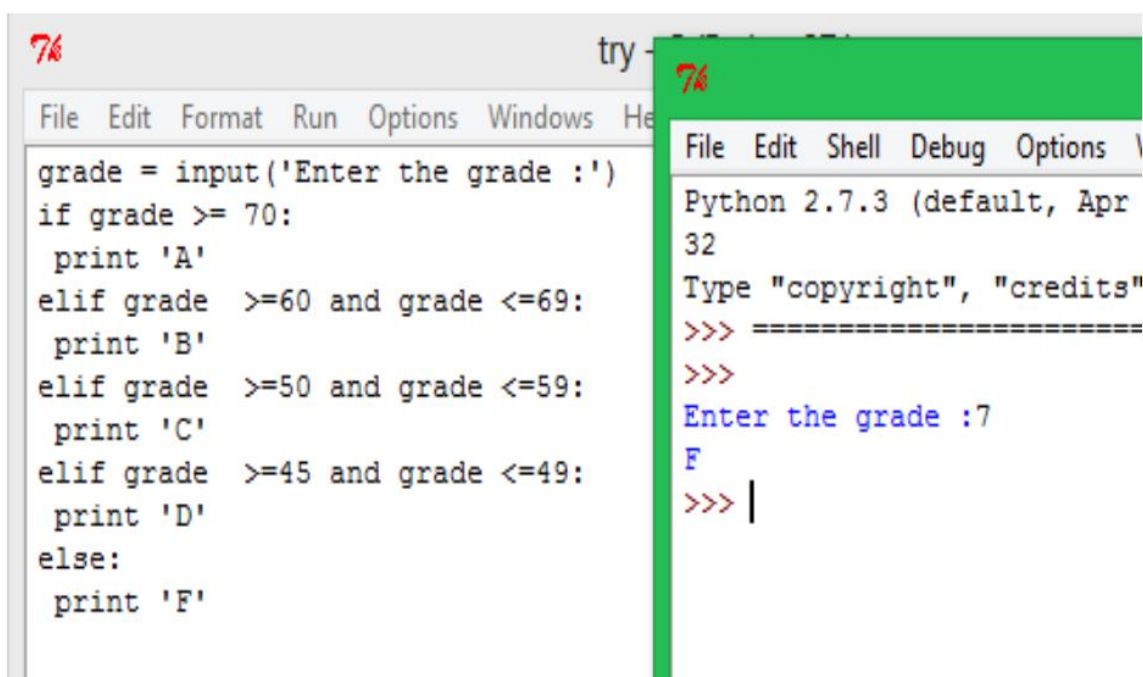
i. ['a', 'd', 'e', 'f']

ii. ['pear', 'apple', 'orange']

iii. [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

iv. [1, 3, 5, 7, 9]

6. a)



The screenshot shows a Python IDE with two windows. The left window, titled '7%', contains the following code:

```
grade = input('Enter the grade :')
if grade >= 70:
    print 'A'
elif grade >=60 and grade <=69:
    print 'B'
elif grade >=50 and grade <=59:
    print 'C'
elif grade >=45 and grade <=49:
    print 'D'
else:
    print 'F'
```

The right window, also titled '7%', shows the execution of the program. It displays the prompt 'Enter the grade :', the user input 'F', and the resulting output 'F'.

d) i. C-like languages require semicolons at the end of each statement, and braces around a nested block of code.

ii. The statements in a nested block are all indented the same number of tabs or spaces.

c)

The following function copies a file, reading and writing up to fifty characters at a time. The first argument is the name of the original file; the second is the name of the new file:

This functions continues looping, reading characters from infile and writing the same charaters to outfile until the end of infile is reached, at which point text is empty and the break statement is executed.

d)

```
sum=0;
i=1
while i<11:|
    i=i+1
    sum=sum+(1/(i*i));
print(sum)
```